

Requirements

python==3.12.7

numpy==1.26.4

matplotlib==2.9.2

pandas==2.2.3

opencv-python==4.11.0.86

jupyter==1.1.1

scikit-learn==1.5.2

seaborn==0.13.2

Github

<https://github.com/rasmusmolby/Ai-data-Miniproject>

Workshop 1

1. Arbejde i grupper på første del af miniprojektet
2. Tilegnelse af et egnet datasæt til arbejde med manglende data (eksempler)
3. Identificer typer af datamangel
4. Implementer/anvend minimum 2 dataimputeringsmetoder til at håndtere manglende data og sammelign (f.eks. ved brug af classifier med/uden imputering)
5. Strukturér data i minimum 2 relationelle databaser (se eksempel fra Forelæsning 2)

I have chosen to work with the Sales and Satisfaction dataset.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 10000 entries, 0 to 9999
Data columns (total 7 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   Group                                8599 non-null   object
1   Customer_Segment                     8034 non-null   object
2   Sales_Before                         8478 non-null   float64
3   Sales_After                         9233 non-null   float64
4   Customer_Satisfaction_Before         8330 non-null   float64
5   Customer_Satisfaction_After         8360 non-null   float64
6   Purchase_Made                       9195 non-null   object
dtypes: float64(4), object(3)
memory usage: 547.0+ KB
Group                                1401
Customer_Segment                     1966
Sales_Before                         1522
Sales_After                         767
Customer_Satisfaction_Before         1670
Customer_Satisfaction_After         1640
Purchase_Made                       805
dtype: int64
```

To identify the type of missing data, I first have to explore the data.

I use `.info()` to see what type of datatype the features are. I then use `.isnull()` to get a better look at how many missing values there are.

```
# Loop through each column that has missing values
for target_col in df:
    print(f"\n Missing pattern for: {target_col}")

    # Create a missing indicator.
    is_missing = df[target_col].isnull()

    # Loop through other columns to check if "missingness" depends on them
    for other_col in df.columns:
        if other_col != target_col:
            print(f"\nGrouped by: {other_col}")

            # If the other column is numeric, bin it first (to make categories)
            if pd.api.types.is_numeric_dtype(df[other_col]):
                binned = pd.qcut(df[other_col], q=4, duplicates='drop')
                grouped = is_missing.groupby(binned, observed=True).mean()
            else:
                grouped = is_missing.groupby(df[other_col], observed=True).mean()

            print(grouped)
```

I use this code to find the missing patterns in each column/feature and compare them with all other features. I use `.qcut()` to sort numerical columns into 4 quantile bins, so it can be compared better with low, mid, mid_high and high values. I then turn each instance into a Boolean (True or False), which tells if the cell has a missing value. This gives me an output of specific numbers, which when observed, can give an indication if any of the missing data is MCAR (Missing Completely At Random) or MAR (Missing At Random).

```

Missing pattern for: Group

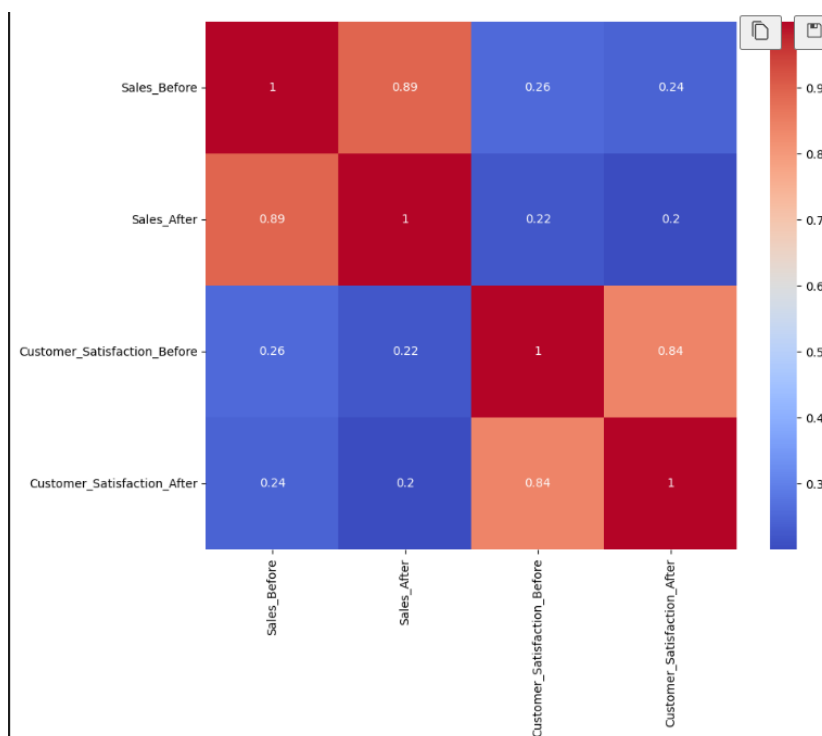
Grouped by: Customer_Segment
Customer_Segment
High Value      0.137604
Low Value       0.147462
Medium Value    0.143864
Name: Group, dtype: float64

Grouped by: Sales_Before
Sales_Before
(24.852, 166.629]    0.148585
(166.629, 203.324]  0.136385
(203.324, 238.809]  0.139217
(238.809, 545.423]  0.141038
Name: Group, dtype: float64

Grouped by: Sales_After
Sales_After
(32.413000000000004, 221.437]    0.152447
(221.437, 273.129]              0.136482
(273.129, 332.57]              0.132149
(332.57, 818.22]               0.144281
Name: Group, dtype: float64

```

This is just a sample, the real output is for each feature and is much longer. These numbers tell me if there are any patterns in the missingness of the data compared to the other features missing data. The numbers need to differ a lot for it to have a high chance of being MAR instead of MCAR, which none of the numbers do, so I can conclude with pretty accurate probability that the missing data is MCAR. Other missing data types would need more domain knowledge, and since this dataset is artificial, it should not be possible to gather that information.



I use a correlation matrix for all the numerical values, to see if there is any good correlation I can use for the imputations. Only things I can indicate is strong correlation between sales_before and sales_after, as they are basically the same thing. Other than that, there does not seem to be that much of a correlation.

```
# Create a non imputed (dirty) and an imputed dataset for the model.
# Also drop na from non imputed so we can compare
df_dirty = df.copy()
imputed_df = df.copy()

# Set target name
target_name = 'Purchase_Made'

# Set up a checker, that only has the not null target feature and not null categorical instances.
# This is instead of using mode, as the performance was worse.
valid_instance = df[categorical_cols].copy().notnull()

# Remove all instances where the target value is missing
df_dirty = df_dirty.dropna(subset=[target_name]).copy()
imputed_df = df.dropna(subset=[target_name]).copy()

# Target data
target_data = df_dirty[target_name]

# KNN Imputation for numerical
knn_imputer = KNNImputer(n_neighbors=5)
imputed_df[numeric_columns] = knn_imputer.fit_transform(imputed_df[numeric_columns])

# Drop target feature in both datasets (And in other relevant elements)
df_dirty = df_dirty.drop(columns=[target_name])
imputed_df = imputed_df.drop(columns=[target_name])
categorical_cols_no_target = categorical_cols.drop(target_name)

# Mode imputation for cat features
#cat_imputer = SimpleImputer(strategy='most_frequent')
imputed_df[categorical_cols_no_target] = cat_imputer.fit_transform(imputed_df[categorical_cols_no_target])
```

To compare my imputed data with the non imputed data, I create 2 copies of the original dataset, one with imputations (imputed_df) and one with none (df_dirty). I choose the “Purchase_Made” feature as my target for the comparison randomly.

I have chosen to use a KNN imputation method for my missing numerical values, as it is better than mean, but still simple, and should suffice with MCAR. I use 5 for N neighbour’s.

For the missing categorical values, I remove the instances outright, as Mode imputation got me worse results. I then use one hot encoding on the rest, so I can use it in my model later.

I split both datasets with a train on 30%. I choose Decision tree classifier to do the predictions, and create 2 models, one for each dataset.

```
=== NON-Imputed (Dirty) Data Model Metrics ===
Accuracy: 0.508
Precision: 0.525
Recall: 0.491
F1 Score: 0.507
ROC AUC: 0.509
Confusion Matrix:
[[587 527]
 [603 582]]

=== Imputed (Clean) Data Model Metrics ===
Accuracy: 0.502
Precision: 0.516
Recall: 0.527
F1 Score: 0.521
ROC AUC: 0.501
Confusion Matrix:
[[529 585]
 [561 624]]
```

The result show that my model did a horrible job at predicting my imputed data and the non imputed data. It is basically the same as random guessing. This indicates that my model did not capture the patterns of either dataset. I could try with other imputation methods, maybe a model based one, but I am guessing since the data is artificial, and the missing data was MCAR, there is not really any pattern in the data. I could be wrong.

TABLES			Customer...#	Sales_Before	Sales_After
Customers			Filter...	Filter...	Filter...
ROWID		1	1	240.5483589001639	300.007567536
CustomerID		2	2	246.8621142097213	381.337554529
Group		3	3	156.97808447859475	179.3304635114
Customer_Segment		4	4	192.1267083410125	229.278031144
Sales		5	5	229.6856225311863	
ROWID		6	6	135.5730030724036	218.5599879367
CustomerID		7	7	191.71391751322253	222.409356291
Sales_Before		8	8	173.75255497726224	213.168231916
Sales_After		9	9	208.3085772	248.178829746
Customer_Satisfac...		10	10	235.07149317447795	352.756872079
Customer_Satisfac...		11	11	139.93153925886617	170.2383962177
Purchase_Made		12	12	NULL	333.064972384
		13	13	211.8349368284123	254.8439121981
		14	14	217.77601332639472	259.989624391
		15	15	173.18344545034157	284.924299421
		16	16	188.33896253003337	232.5824857981
		17	17	306.7014523489638	485.135424491

For the rational databases, i used SQLite to create a .db file with 2 custom tables. I used the unfiltered raw data to do this, as my imputation was bad, so it would not matter.

1. A table for the customer, which has a CustomerID that gets placed at each instance, and also the "Group" and "Customer_Segment".
2. A table for everything "sales" related. This includes the Customer ID, sales_before, sales_after, customer_satisfaction_before, customer_satisfaction_after and purchase made. The file can be found in the github folder.

Workshop 2

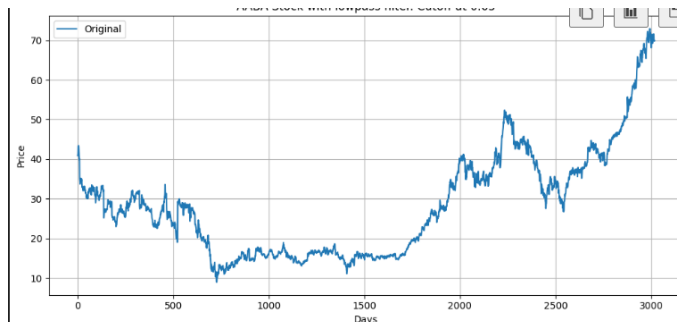
1. Arbejde i grupper på anden del af miniprojektet.
2. Tilegnelse af et egnet datasæt til arbejde med støjfuld data (eksempler)
3. Vurder graden af støj for den eller de tidsseriedata der er i datasættet gennem datavisualisering.
4. Anvend et passende præprocessering (moving average, eller frekvensdomænefiltrering).
5. Plot data før og efter præprocessering.

I have chosen to use "DJIA 30 Stock Time Series" created by SZRLEE as my primary dataset for the workshop 2 project.

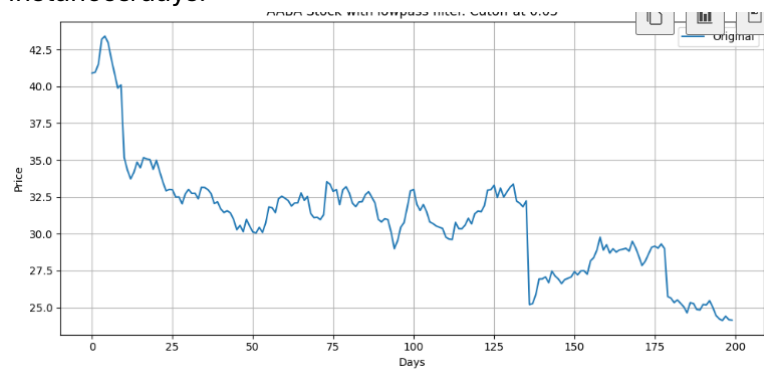
From this file I have chosen to only focus on the stock AABA in the time-zone 2006-01-01 to 2018-01-01 for a more focused perspective.

Therefore, i will only use the file AABA_2006-01-01_to_2018-01-01.csv.

First, I start by getting a visualization of the data by using matplotlib.



Here we see a lot of sudden spikes. I zoom in to get a better look by taking the first 200 instances/days.



There are many sudden drop-offs, which will make it hard to use the data for real world applications. For example, if I wanted to see how the stocks general movement was in a time zone, its going to be a problem when there is a lot of noise in the dataset. If I were to differentiate day 27, the change would be a couple of cents, but if I were to do the same at day 135 (the big drop in the picture above), there would be a change of multiple dollars.

To get a better understanding of the stock, and how it changes overtime, I am going to use Fourier filtering to remove some of the noise, and smooth out the graph.

First i download the data from kaggle, load it in using pandas. I focus only on the “close” column, as it is the closing price of that day.

I centre the graph around the x-axis, for a better transformation. This is done by taking the mean value of all the instances and subtracting it from the original.

```
# Fourier transformation. I use FFT (Fourier Fast Transformation) which is  
# a more effecient method for doing Discrete Fourier Transformation  
fft_values = np.fft.fft(price_closed)  
frequencies = np.fft.fftfreq(len(price_closed))
```

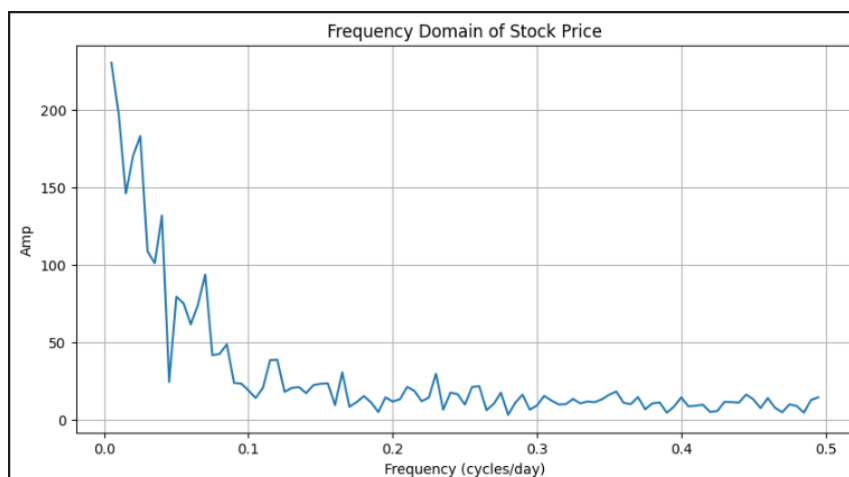
```
# Making sure that we only get the positive values.  
pos_only = frequencies > 0  
amp = np.abs(fft_values[pos_only])  
freqs_pos = frequencies[pos_only]
```

✓ 0.0s

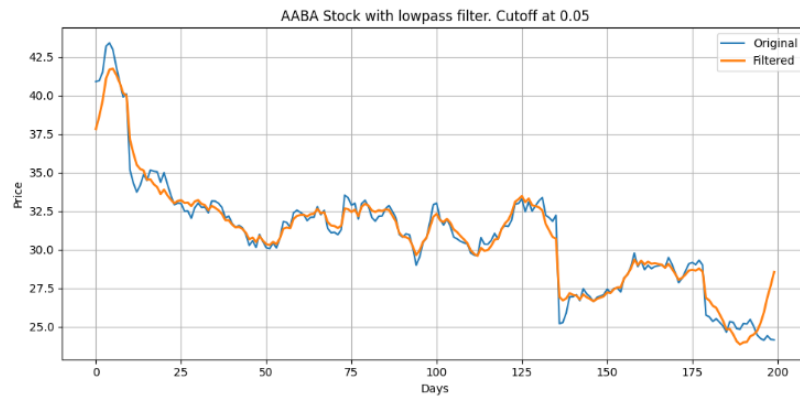
I the use Fourier transformation on the centered data, to get frequencies and the values. I also make sure it is only positive numbers.

```
# Choose low-pass to reduce the noise.  
cutoff = 0.05 # Chosen from looking at frequency domain, was a aggresive dropoff  
filtered_fft = fft_values.copy()  
filtered_fft[np.abs(frequencies > cutoff)] = 0 # Filter (low pass)
```

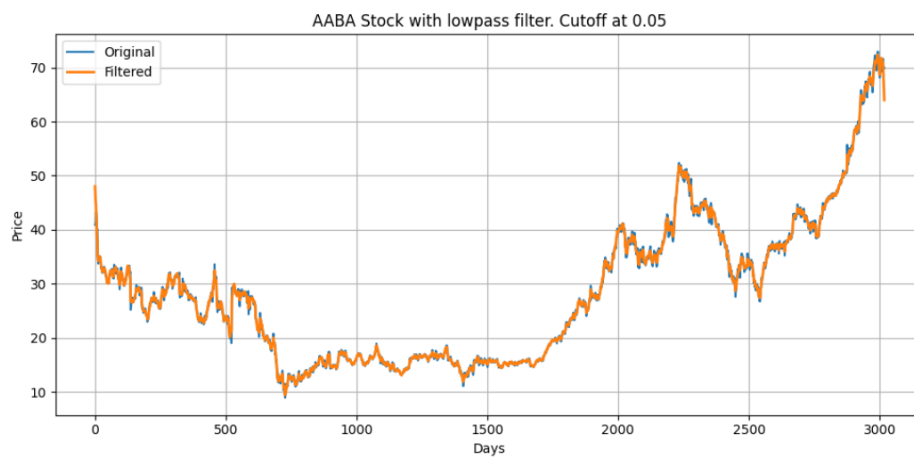
I then plot the frequency domain, so I can select a relevant cutoff point for my filter. I chose to use lowpass filter, as it cuts off the high fluctuating frequencies, which is what im am trying to do.



By looking at the graph, we can determine the best cutoff would be in the 0.02-0.05, where the steepest fluctuations happen. I have tried with 0.01-0.05 cutoffs for my data, and by visually observing, concluded that 0.05 is a good cutoff value, as it smooths out the data, but keeps the patterns.



Data with a lowpass filter and a cutoff value at 0.05. Only in the range from 0-200 days. The data is significantly reduced in noise from the original, but still has the overall patterns.



Here is the whole AABA data, with original and filtered values.